BSc (Hons) in **Computer Science**

Faculty of Computing

SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Practical 01

> **Objective & Lecture Mapping:** Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the Git Hub Repo. Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance measure
Environment
Actuators
Sensors

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

self.width / self.height - Grid dimensions (size of the world).
self.agent_pos - Starting location of the primary agent.
self.walls - Set of impassable wall coordinates.
self.food_positions - Set of food-item coordinates.
self.opponents - List of opponent-agent coordinates (other agents in the world).
self.toxic_traps - Set of trap coordinates (hazard cells).
self.score, self.steps, self.collision - Global state that reflects the outcome of interactions with the environment.

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Multi Agent
The presence of self.opponents (a list of other agents that move independently) means the environment contains more than one autonomous entity. Even if num_opponents is set to 0, the design supports multiple agents, so the baseline

Task 1.2: The Perception Subsystem (get_percept)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the ordered list of percepts that an agent receives over time:

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

get_percept() implements the Sensors. It translates the underlying environment state into a percept dictionary that the agent can observe.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

The agent does not see the full layout of walls, the exact locations of all food items (only a boolean “smells_food”), or the full set of traps (only a boolean when standing on one). Because the agent lacks complete information about the entire world state,

Task 1.3: Action Execution (execute_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

The line self.score -= 5 updates the Performance measure component. The score is the metric used to evaluate how well the agent is doing.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

A performance measure should reflect the agent’s actual task goals like collecting food or avoiding hazards rather than irrelevant internal effort. External state changes (e.g., hitting a wall) provide fair comparisons across agents with different algorithms or hardware, while reinforcement learning requires rewards tied to observable consequences for clear causal signals. Separating computation cost from outcome cost keeps the design clean: sensors feed perception, actions affect the environment,

Part 2: Guided Modification & Deep Theory Mapping

> Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (__init__)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally hide this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

Since the agent's sensors (get_percept) do not detect toxic traps or report when one is triggered, the environment remains partially observable. The agent may lose points without knowing why and cannot identify or avoid trap locations, forcing it to

Step 2.2: Updating the Perception Subsystem (get_percept)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

An agent that receives real-time sensory input about hazards can directly link those signals to sharp drops in expected utility, enabling it to adjust its policy to avoid or escape harmful states. By exposing hidden dangers through sensors, the environment shifts toward fuller observability, allowing the agent to act rationally

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

A vacuum agent rewarded only for sucking up dirt can exploit its metric by repeatedly dumping and re-cleaning the same dirt, maximizing points while failing the true goal of keeping the floor clean. This highlights the danger of defining performance measures around intermediate actions rather than desired environmental states. To prevent such exploits, metrics must penalize undesirable

Finalize and Lock Lab 01 Analytical Evaluation

Lab 01 code analysis and theoretical evaluation complete and verified!

FACULTY OF COMPUTING